COMS W4167: Computer Animation

Programming Assignment 3

Due 10:00 PM Saturday, Mar. 14th, 2026 (EST)

Academic Honesty Policy

You are permitted and encouraged to discuss your work with other students. Except where explicitly stated otherwise, you may work out equations in writing on paper or a whiteboard. You are encouraged to use Ed Discussion to converse with other students, the TAs, and the instructor.

HOWEVER, you must NOT share source code or hard copies of source code. Refrain from activities or the sharing materials that could cause your source code to APPEAR TO BE similar to another student's enrolled in this or previous years. We will be monitoring source code for individuality. Cheating will be dealt with severely. Source code should be yours and yours only. Do not cheat. For more details, please refer to the full academic honesty policy on the Engineering School's website.

Chapter 1

Course Notes

1.1 Primitive Collision Detection

In the class, we discussed various general collision detetction algorithms using bounding volumne hierarchies (BVHs). Since we haven't formally discussed rigid bodies, which are objects with arbitrary shapes, we outline here basic collision detections that you will need to finish the programming assignments.

Suppose you have two primitive objects (each of which can be a particle or a half-plane) O_1 and O_2 with thicknesses r_1 and r_2. Conceptually, the algorithm for checking whether or not O_1 and O_2 are colliding is to

1. Consider all vectors between points in O_1 and O_2,
2. Find the vector $\mathbf{n}$ that is the shortest,
3. Compare its length $|\mathbf{n}|$ to $r_1 + r_2$. If $\mathbf{n}$ has length less than $r_1 + r_2$, the objects are overlapping.
4. Lastly, check if they are approaching or moving apart. If they are approaching, then the objects are colliding.

Figure **??** illustrates this algorithm for two particles.

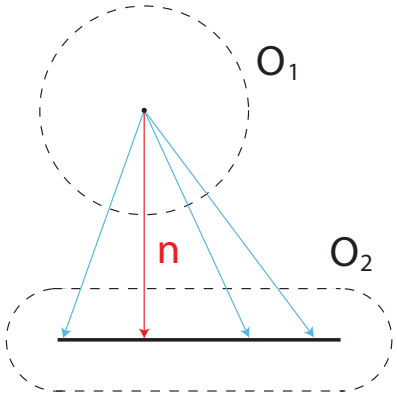

Figure 1.1: Checking whether or not a particle and an edge are colliding. Several vectors between the particle and and the edge are drawn in blue; the shortest one $\mathbf{n}$ is drawn in red. Since $\mathbf{n}$ has length greater than the sum of the two objects' radii, the objects are not overlapping and so are not colliding.

Although this algorithm gives a good overview of the big picture, it cannot be practically implemented as written above: for instance, you wouldn't want to write code to actually find the set of all possible vectors between points in O_1 and O_2. Instead, by examining the geometry of the colliding objects, we will derive formulas for the shortest vector, skipping the first step entirely.

The types of collisions that can occur are particle–particle and particle–half-plane. Following this idea, you can derive formulas for other types of primitives, such as cylinders, but those are beyond the scope of this assignment.

Particle–Particle

Finding the shortest vector from O_1 to O_2 is trivial when both objects are particles: there is only one vector to choose from. When checking the length of $\mathbf{n}$, don't forget that the two particles might have different radii.

To check if the particles are approaching, look at the difference in velocity along the $\mathbf{n}$ direction, $(\mathbf{v}_1 - \mathbf{v}_2) \cdot \mathbf{n}$. This scalar is proportional to the speed at which the particles are moving toward each other; if it is positive, then the particles are approaching.

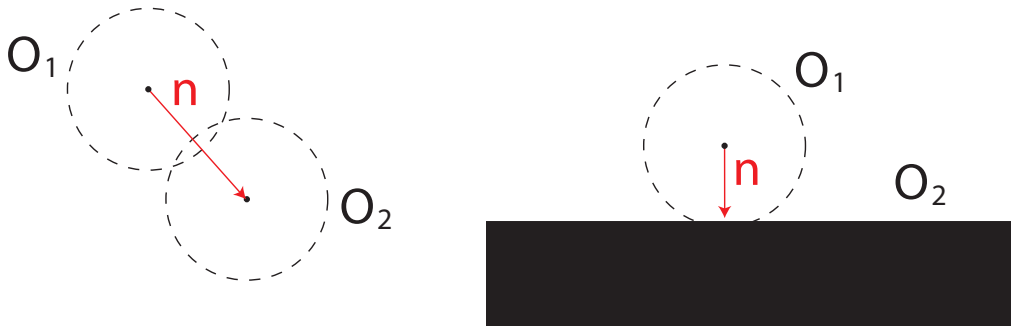

Figure 1.2: Left: A particle–particle collision. Right: A particle–half-plane collision. In each case, the shortest vector between the objects, $\mathbf{n}$, has been drawn in red.

Particle–Half-plane

Let $\mathbf{x}_h$ and $\mathbf{n}_h$ be the position and normal of the half-plane. The shortest vector between the particle and half-plane has direction $-\mathbf{n}_h$ and magnitude equal to the distance between the particle and boundary of the half-plane. This distance is equal to

$$\frac{(\mathbf{x}_h - \mathbf{x}) \cdot -\mathbf{n}_h}{\|\mathbf{n}_h\|},$$

so

$$\mathbf{n} = \frac{(\mathbf{x}_h - \mathbf{x}) \cdot \mathbf{n}_h}{\|\mathbf{n}_h\|^2}\mathbf{n}_h. \tag{1.1}$$

The velocity of the particle in the direction $\mathbf{n}$ is proportional to $\mathbf{v} \cdot \mathbf{n}$. If this quantity is positive, the particle is approaching the half-plane.

Note. Here we don't assume $\mathbf{n}_h$ is noramlized so we devide it by $\|\mathbf{n}_h\|$. Also note that in the scene specification file (i.e., .YMAL files), the plane is specified by $\mathbf{n}_h$ and an offset c:

```
- type: half_space_plane
  normal: [0.3, 0.0, 1]
  offset: 1.2
```

The offset c defines $\mathbf{x}_h$ as $\mathbf{x}_h = c\frac{\mathbf{n}_h}{\|\mathbf{n}_h\|}$.

Automatic Testing of Detection by the Oracle

To aid you in debugging your code, we have added functionality to the oracle to automatically compare the collisions you found against those the oracle is expecting. When running the oracle in binary input mode, during each frame it will show you:

1. Missed collisions: those that the oracle detected this frame, but weren't detected by your code. The shortest vector $\mathbf{n}$ between the two objects whose collision was missed is drawn in green to indicate the missed collision.

2. Superfluous collisions: those that your code detected, but that the oracle didn't. The vector $\mathbf{n}$ of the superfluous collision is drawn on top of the simulation in red.

3. Incorrect $\mathbf{n}$: both the oracle and your code agree that a collision occurred, but disagree on the shortest vector $\mathbf{n}$. The incorrect vector is drawn in red, and the correct one in green.

1.2 Overview of Contact Response

We discussed two kinds of contact response, *impulse-based response* and *penalty-based response*. Let's start with the impulse-based response. Once we detect a collision, we must apply *impulses*—instantaneous changes to momentum—to *respond* to the collision.

1.2.1 Impulse Response

Particle–Particle

Denoting post-collision-response velocities with tildes, we apply an impulse to the velocities of the particles O_1 and O_2 using the equations

$$\tilde{\mathbf{v}}_1 \leftarrow \mathbf{v}_1 + \mathbf{I}_1/m_1 \tag{1.2}$$

$$\tilde{\mathbf{v}}_2 \leftarrow \mathbf{v}_2 + \mathbf{I}_2/m_2 \tag{1.3}$$

for some as-yet-undetermined impulse vectors $\mathbf{I}_1$ and $\mathbf{I}_2$. By choosing these impulse vectors carefully, we stop the particles from approaching any further, while obeying the laws of physics.

Denote by $\hat{\mathbf{n}}$ the unit vector in direction $\mathbf{n}$. $\hat{\mathbf{n}}$ is called the *contact normal*. Pushing the objects apart in the direction of this vector most quickly increases the distance between them, so we want the impulses $\mathbf{I}_1$ and $\mathbf{I}_2$ to lie in the same direction as $\hat{\mathbf{n}}$:

$$\mathbf{I}_1 = I_1\hat{\mathbf{n}}$$
$$\mathbf{I}_2 = I_2\hat{\mathbf{n}}$$

for scalars I_1 and I_2.

To find formulas for these scalars, we write down the laws of conservation of momentum and energy. Conservation of momentum tells us that the total momentum of the two objects before and after we apply the impulses should be equal. Therefore

$$\begin{aligned}
m_1\mathbf{v}_1 + m_2\mathbf{v}_2 &= m_1\tilde{\mathbf{v}}_1 + m_2\tilde{\mathbf{v}}_2 \\
m_1(\mathbf{v}_1 - \mathbf{v}_1 - \mathbf{I}_1/m_1) &= m_2(\mathbf{v}_2 + \mathbf{I}_2/m_2 - \mathbf{v}_2) \\
-\mathbf{I}_1 &= \mathbf{I}_2 \\
-I_1\hat{\mathbf{n}} &= I_2\hat{\mathbf{n}} \\
-I_1\hat{\mathbf{n}}\cdot\hat{\mathbf{n}} &= I_2\hat{\mathbf{n}}\cdot\hat{\mathbf{n}} \\
-I_1 &= I_2.
\end{aligned}$$

Similarly, energy before and after applying the impulse must be equal. In particular, since applying an impulse only modifies velocities and not positions, and so doesn't change potential energy, kinetic energy before and after the impulse must be equal:

$$\begin{aligned}
\frac{1}{2}m_1\mathbf{v}_1\cdot\mathbf{v}_1 + \frac{1}{2}m_2\mathbf{v}_2\cdot\mathbf{v}_2 &= \frac{1}{2}m_1\tilde{\mathbf{v}}_1\cdot\tilde{\mathbf{v}}_1 + \frac{1}{2}m_2\tilde{\mathbf{v}}_2\cdot\tilde{\mathbf{v}}_2 \\
m_1\mathbf{v}_1\cdot\mathbf{v}_1 + m_2\mathbf{v}_2\cdot\mathbf{v}_2 &= m_1\mathbf{v}_1\cdot\mathbf{v}_1 + 2I_1\mathbf{v}_1\cdot\hat{\mathbf{n}} + I_1^2/m_1\hat{\mathbf{n}}\cdot\hat{\mathbf{n}} \\
&\quad + m_2\mathbf{v}_2\cdot\mathbf{v}_2 + 2I_2\mathbf{v}_2\cdot\hat{\mathbf{n}} + I_2^2/m_2\hat{\mathbf{n}}\cdot\hat{\mathbf{n}} \\
0 &= 2I_1\mathbf{v}_1\cdot\hat{\mathbf{n}} + 2I_2\mathbf{v}_2\cdot\hat{\mathbf{n}} + I_1^2/m_1 + I_2^2/m_2.
\end{aligned}$$

Since we know $I_2 = -I_1$,

$$\begin{aligned}
0 &= 2I_1\mathbf{v}_1\cdot\hat{\mathbf{n}} - 2I_1\mathbf{v}_2\cdot\hat{\mathbf{n}} + I_1^2/m_1 + I_1^2/m_2 \\
0 &= 2(\mathbf{v}_1 - \mathbf{v}_2)\cdot\hat{\mathbf{n}} + I_1/m_1 + I_1/m_2 \\
I_1(m_1 + m_2) &= 2m_1m_2(\mathbf{v}_2 - \mathbf{v}_1)\cdot\hat{\mathbf{n}} \\
I_1 &= \frac{2(\mathbf{v}_2 - \mathbf{v}_1)\cdot\hat{\mathbf{n}}}{\frac{1}{m_1} + \frac{1}{m_2}}.
\end{aligned}$$

Therefore the final formulas for updating velocities when a collision is detected are

$$\begin{aligned}\tilde{\mathbf{v}}_1 &\leftarrow \mathbf{v}_1 + \frac{2(\mathbf{v}_2 - \mathbf{v}_1)\cdot\hat{\mathbf{n}}}{1 + \frac{m1}{m_2}}\hat{\mathbf{n}} \\ \tilde{\mathbf{v}}_2 &\leftarrow \mathbf{v}_2 - \frac{2(\mathbf{v}_2 - \mathbf{v}_1)\cdot\hat{\mathbf{n}}}{\frac{m_2}{m_1} + 1}\hat{\mathbf{n}}.\end{aligned} \tag{1.4}$$

Note. For fixed particles, its effective mass is infinity. For example, if the first particle is fixed, in (**??**), $\frac{m_2}{m_1} = 0$, and only $\mathbf{v}_2$ is updated, and $\mathbf{v}_1$ stays zero.

Particle–Half-Plane

Since the half-plane is fixed in place, we only have a particle to which we must apply an impulse:

$$\tilde{\mathbf{v}} \leftarrow \mathbf{v} + I/m\hat{\mathbf{n}},$$

where $\hat{\mathbf{n}}$ is the normal of the plane (pointing outward of the plane). Conservation of energy tells us the right value of I:

$$\begin{aligned}\frac{1}{2}m\tilde{\mathbf{v}}\cdot\tilde{\mathbf{v}} &= \frac{1}{2}m\mathbf{v}\cdot\mathbf{v} \\ 2I/m\,\mathbf{v}\cdot\hat{\mathbf{n}} + I^2/m^2\,\hat{\mathbf{n}}\cdot\hat{\mathbf{n}} &= 0 \\ I/m^2 &= -2/m\,\mathbf{v}\cdot\hat{\mathbf{n}} \\ I &= -2m\,\mathbf{v}\cdot\hat{\mathbf{n}}.\end{aligned}$$

The final formula for the change in the velocity of the particle is thus

$$\tilde{\mathbf{v}} \leftarrow \mathbf{v} - 2(\mathbf{v}\cdot\hat{\mathbf{n}})\hat{\mathbf{n}}.$$

Fixed Particles

Fixed particles and edges require special consideration. On the one hand, we don't want to apply any impulses to them that will change their velocity, since they are supposed to be fixed in place. On the other, we can't ignore them completely during collision detection and response, since we do want non-fixed particles and edges to interact with them.

Since changes in velocity are equal to impulse divided by mass, setting the mass of fixed particles to infinity is one way of ensuring that they are "immovable objects" during collisions. Implement this handling of fixed objects by doing the following:

- Instead of applying an impulse to a fixed particle, do nothing instead. (This step is necessary to avoid $\frac{\infty}{\infty}$ in certain degenerate cases.)
- Any time you would use the mass of a fixed particle while applying an impulse to a non-fixed particle, use infinity for that mass instead.

Coefficient of Restitution

When you drop a ball on the ground, conservation of energy suggests that the ball must bounce back up to exactly its original height. Of course, balls in the real world do not do this: bowling balls barely bounce up at all, and even the bounciest rubber ball only returns to a fraction of its original height.

Many complicated factors contribute to this dissipation of energy: plastic deformation, internal friction during elastic deformation, and production of sound waves are just a few examples. Simulating all of these small-scale phenomena is not feasible, so instead many simulations approximate the net coarse-scale behavior of these effects by a *coefficient of restitution* (COR). There are multiple ways of defining the COR, and we

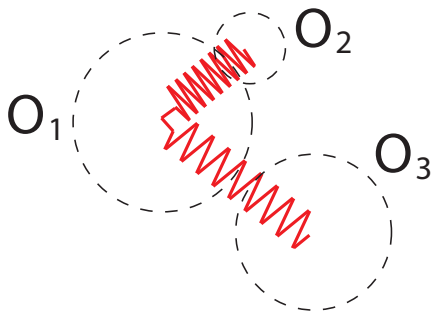

Figure 1.3: A conceptual illustration of the penalty method. A spring-like penalty force repels O_1 and O_2, since they are interpenetrating, and O_1 and O_3, since they are sufficiently close to each other (less than distance T apart, where T is the penalty method thickness). No force is exerted between O_2 and O_3 because they are far apart.

adopt the *Newtonian* COR as discussed in class (note: this is not the COR which is the ratio of the height of bounce to the dropping height).

Although the derivation is beyond the scope of this course, it turns out that *Newtonian* COR can be simulated by multiplying all impulses applied during contact response by the following scalar:

$$\frac{1+\text{COR}}{2}.$$

COR should be in the range of $[0, 1]$. Notice that when COR is 1.0, the impulses don't change; this is as expected, since COR=1.0 corresponds to perfect conservation of energy.

Note. In the assignment code, COR can be specified for each half-plane and particle individually. For example,

```
fixed_shapes:
- type: half_space_plane
  normal: [0.0, 0.2, 1]
  offset: 0.0
  restitution: 0.6
```

When two objects collide, one with a COR c_1 and another with a COR c_2, we need to choose one COR to compute the impulse between these two objects. We choose the one that dissipate the most energy, i.e., using $c = \min(c_1, c_2)$. This is the convention that many simulation engines use.

1.2.2 Penalty Method

The main idea behind the penalty method is simple: for every pair of objects in the simulation, we add a new force—a *penalty force*—that does nothing if the objects are far apart. If the objects are close or colliding, on the other hand, the force acts like a spring, pushing them apart. The closer the two objects get, the stronger the force becomes, and the harder the simulation tries to push them apart. Figure **??** illustrates the penalty method conceptually.

For two objects O_1 and O_2 of radius r_1 and r_2, their penalty potential is given by the formula

$$V = \begin{cases} 0, & \|\mathbf{n}\| > r_1 + r_2 \\ \frac{1}{2}k(\|\mathbf{n}\| - r_1 - r_2)^2, & \|\mathbf{n}\| \leq r_1 + r_2, \end{cases} \tag{1.5}$$

where k is a penalty force stiffness, and $\mathbf{n}$ is the usual shortest vector between O_1 and O_2. Notice that this potential bears a lot of similarity to that for a spring, given in previous assignments, with two notable differences: firstly, the potential is constant (0) whenever $\|\mathbf{n}\| > r_1 + r_2$, which holds whenever the objects are not in contact. Since force is the gradient of potential, a constant potential when the objects aren't penetrating translates into zero force when the objects are far apart. Secondly, the length of the spring has been replaced with the length of the shortest vector between the objects, and the rest length has been replaced with the sum of the objects' radii.

The penalty method has several pros and cons when compared to detecting and applying impulses. As you will see, it is quite easy to implement: it's just another force you add to the simulation, without needing to do any separate collision detection or modification of the main simulation loop. On the other hand, the penalty force requires that you choose a value for the spring stiffness k: set it too high and the simulation becomes unstable, unless you decrease the time step and thereby slow down the simulation. Set it too low and the penalty force is very weak, resulting in a lot of "give" when objects collide, and in the worst case allowing objects to tunnel completely through each other.

As usual, the penalty force is derived from the potential by taking the gradient with respect to position:

$$F = -(\nabla V)^T = \begin{cases} 0, & \|\mathbf{n}\| > r_1 + r_2 \\ -k(\|\mathbf{n}\| - r_1 - r_2)(\nabla \mathbf{n})^T \hat{\mathbf{n}}, & \|\mathbf{n}\| \leq r_1 + r_2, \end{cases}$$

where the precise formula for the rectangular matrix $\nabla \mathbf{n}$ depends on the two objects involved (particle–particle or particle–half-plane) Formulas for each case are derived below.

Particle–Particle

For a pair of particles, $\mathbf{n} = \mathbf{x}_2 - \mathbf{x}_1$, where $\mathbf{x}_1$ and $\mathbf{x}_2$ are the positions of the first and second particle, respectively, so

$$\nabla \mathbf{n} = \begin{bmatrix} -\mathbf{I} & \mathbf{I} \end{bmatrix}.$$

Important Note: This matrix is expressed in local indices, with the left block corresponding to the degrees of freedom for the first particle, and the right block corresponding to those for the second particle.

Particle–Half-plane

From (**??**), we have that

$$\mathbf{n} = \frac{(\mathbf{x}_h - \mathbf{x}) \cdot \mathbf{n}_h}{\|\mathbf{n}_h\|^2} \mathbf{n}_h,$$

and thus obtain

$$\nabla \mathbf{n} = -\frac{\mathbf{n}_h \mathbf{n}_h^T}{\|\mathbf{n}_h\|^2}.$$

1.3 Continuous Collision Detection

So far, when we respond collisions, we freeze the time and detect collisions. As discussed in class, the approach suffers from tunneling, and we need to resort to continuous collision detection to avoid tunneling (see Fig. **??**).

Suppose we have the positions of objects at the start of a time step ($\mathbf{q}^s$) and the end of the time step ($\mathbf{q}^e$). We assume that the particles and edges moved in straight lines between the two time steps; in other words, the positions between the two time steps are given by the continuous function $\mathbf{q}(t) = \mathbf{q}^s + t\Delta\mathbf{q}$, where $\Delta\mathbf{q} = \mathbf{q}^e - \mathbf{q}^s$. The parameter t can be thought of as the time since the start of the time step, where for convenience time has been renormalized so that the time step has duration 1.

To prevent tunneling, we must ask, **"is there any time t in $[0,1]$ when the two objects are (a) overlapping, and (b) approaching?"** Answering this question is called performing *continuous-time collision detection*, since we are looking at all values of t instead of just $t = 1$.

For each pair of objects O_1, O_2 (with radii r_1 and r_2) we derived formulas for the shortest vector $\mathbf{n}$ between them (recall Sec. **??**). These formulas depended only on the beginning-of-time-step positions $\mathbf{q}^e$ of the objects; we can thus write down the shortest vector $\mathbf{n}(t)$ as a function of time by replacing $\mathbf{q}^e$ with $\mathbf{q}(t) = \mathbf{q}^s + t\Delta\mathbf{q}$ in these formulas. Question (a) then boils to, "is there some time t in $[0,1]$ when $|\mathbf{n}(t)| < r_1 + r_2$?" Similarly, question (b) can be rephrased as, "is there some time t in $[0,1]$ when the relative velocity $\mathbf{r}(\Delta\mathbf{q})$ of the two objects along $\mathbf{n}(t)$ is positive?"

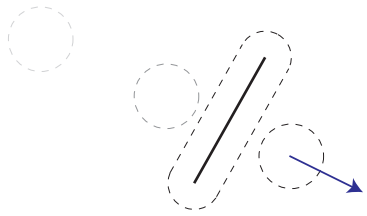

Figure 1.4: Simply checking for collisions at the end of the time step misses some collisions, such as this situation where a particle with high velocity *tunnels* through an edge.

Performing continuous-time collision detection is thus equivalent to checking whether the *simultaneous system of inequalities*

$$\begin{aligned} \sqrt{\mathbf{n}(t) \cdot \mathbf{n}(t)} &< r_1 + r_2 \\ \mathbf{r}(\Delta\mathbf{q}) \cdot \mathbf{n}(t) &> 0 \end{aligned}$$

has a solution between $t = 0$ and $t = 1$. Solving such systems of inequalities is in general very hard. One thing we can do to make our lives a little bit easier is to square both sides of the first inequality, getting rid of the square root; this squaring is permitted since both sides of the inequality must be positive.

$$\begin{aligned} \mathbf{n}(t) \cdot \mathbf{n}(t) &< (r_1 + r_2)^2 \\ \Delta\mathbf{q} \cdot \mathbf{n}(t) &> 0. \end{aligned}$$

For particle–particle and particle–halfplane collisions, these equations can be written as *polynomial* inequalities, which are much easier to solve than general non-linear inequalities.

1.3.1 Systems of Polynomial Inequalities

First, I'd like to discuss the general idea of solving a system of polynomial inequalities. Suppose we have some polynomials $f(t), g(t)$ and we want to find a simultaneous solution to

$$\begin{aligned} f(t) &\geq 0 \\ g(t) &\geq 0. \end{aligned} \tag{1.6}$$

Here's the algorithm. We take one of our polynomials, f, and find all of its roots (e.g., using numpy.roots method). We know that between consecutive roots, f must have the same sign (positive or negative), since f's sign can only change at a root. By evaluating $f(t)$ at any value of t between the two roots we can determine this sign.

We can thus find the *intervals* of t on which $f(t)$ is positive. For example, the polynomial $x^2 - 1$ has two intervals on which it is positive: $(-\infty, -1)$, and $(1, \infty)$. $-x^2 + 1$ has one: $(-1, 1)$. Higher-degree polynomials might have many such intervals.

We can then do the same thing for $g(t)$. If any of f's intervals overlap one of g's intervals, then any value of t inside the *intersection* of the two intervals is a solution to the system of inequalities. Figure **??** illustrates this algorithm for two example polynomials.

As a final note, although this algorithm has been described for two polynomials, it is easy to extend to the problem of solving for when an arbitrary number of polynomials $f(t), g(t), h(t), \ldots$ are simultaneously positive.

Note. In pratice, we need to find the *earliest* time t_c in $[0,1]$ such that (**??**) is satisfied if any. We will respond with an impulse according to the normal direction $\mathbf{n}(t_c)$ at t_c, as described next.

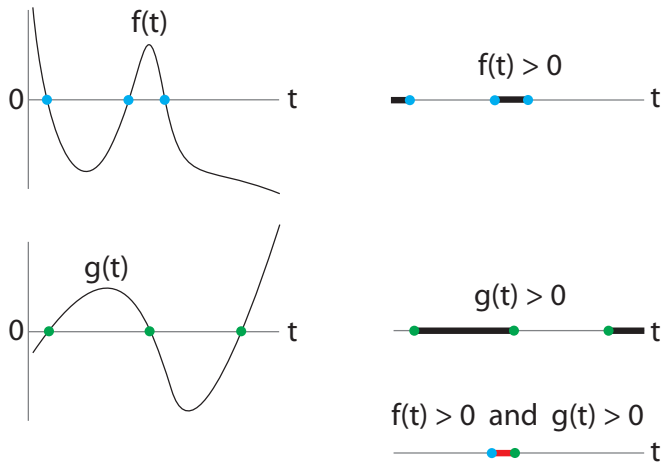

Figure 1.5: Solving the system of inequalities $f(t) > 0, g(t) > 0$ for two example polynomials. Left: Plots of the two polynomials. Right: The intervals of t on which f and g are positive, and the intersection (bottom, in red) of the two sets of intervals. Any point inside a red interval is a solution to the system of inequalities.

1.3.2 Responding Continuous-Time Collision Detection with Impulses

Using the above polynomial inequality algorithm, we can determine whether or not two objects collide during a time step. But what do we do with this information? Conceptually, we partition time-stepping into two parts. First, we step the simulation forward using whatever integrator has been specified for the simulation. This gives us *predicted* positions and velocities $\mathbf{q}^e$ and $\dot{\mathbf{q}}^e$. We then perform continuous-time collision detection and response, modifying $\mathbf{q}^e$ and $\dot{\mathbf{q}}^e$ to prevent all collision during the time step. Here is the algorithm in detail:

1. Step the old positions $\mathbf{q}^s$ and velocities $\dot{\mathbf{q}}^s$ forward using the numerical integrator to get predicted new positions $\mathbf{q}^e$ and velocities $\dot{\mathbf{q}}^e$.

2. Assume the particles moved with constant velocity from $\mathbf{q}^s$ to $\mathbf{q}^e$, and perform continuous-time collision detection with $\mathbf{q}(t) = \mathbf{q}^s + t\Delta\mathbf{q}$, where $\Delta\mathbf{q} = \mathbf{q}^e - \mathbf{q}^s$.

3. If any collisions were detected, apply impulses to the predicted velocities $\dot{\mathbf{q}}^e$ to get modified velocities $\dot{\mathbf{q}}^m = \dot{\mathbf{q}}^e + \mathbf{M}^{-1}\mathbf{I}$. The vector $\mathbf{I}$ is the impulse derived in Sec. **??** needed to resolve the collision. When computing the impulses, the contact normal direction should be the normal direction at the earliest contact time (i.e., $\mathbf{n}(t_c)$ described above).

4. Also modify positions to get new collision-free positions: $\mathbf{q}^m = \mathbf{q}^e + h\mathbf{M}^{-1}\mathbf{I}$.

Note that this algorithm assumes that applying impulses does not cause any new collisions. We will examine this assumption and further improve collision response with the hybrid method later this course note.

Next, we derive formulas needed for continuous collision detection for particle–particle and particle–half-plane. Notice that what we described above is a general algorithm for continuous collision detection. When it comes to the specific cases for particle–particle and particle–half-plane, the implementation can be significantly simplified.

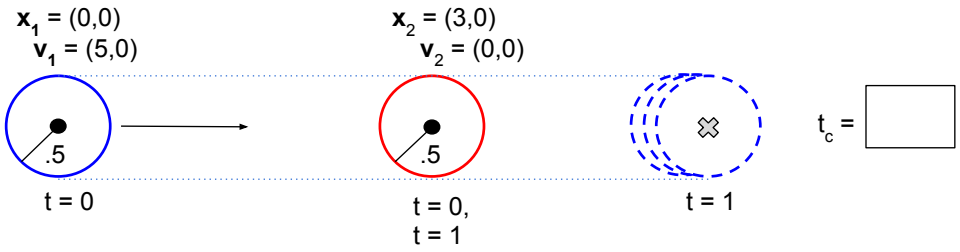

Figure 1.6: As a useful debug tool whenever deriving or confirming polynomials, try to set up example scenes where the answer is known and verifying your results.

Particle–Particle

For two particles with centers at $\mathbf{x}_1(t)$ and $\mathbf{x}_2(t)$, the vector $\mathbf{n}(t)$ is simply $\mathbf{x}_2(t) - \mathbf{x}_1(t)$. The first polynomial we need is therefore

$$\begin{aligned}
\mathbf{n} \cdot \mathbf{n} &< (r_1 + r_2)^2 \\
(\mathbf{x}_2^s + t\Delta\mathbf{x}_2 - \mathbf{x}_1^s - t\Delta\mathbf{x}_1) \cdot (\mathbf{x}_2^s + t\Delta\mathbf{x}_2 - \mathbf{x}_1^s - t\Delta\mathbf{x}_1) &< (r_1 + r_2)^2 \\
[(\mathbf{x}_2^s - \mathbf{x}_1^s) + t(\Delta\mathbf{x}_2 - \Delta\mathbf{x}_1)] \cdot [(\mathbf{x}_2^s - \mathbf{x}_1^s) + t(\Delta\mathbf{x}_2 - \Delta\mathbf{x}_1)] &< (r_1 + r_2)^2 \\
(\mathbf{x}_2^s - \mathbf{x}_1^s) \cdot (\mathbf{x}_2^s - \mathbf{x}_1^s) + 2t(\mathbf{x}_2^s - \mathbf{x}_1^s) \cdot (\Delta\mathbf{x}_2 - \Delta\mathbf{x}_1) + t^2(\Delta\mathbf{x}_2 - \Delta\mathbf{x}_1) \cdot (\Delta\mathbf{x}_2 - \Delta\mathbf{x}_1) &< (r_1 + r_2)^2,
\end{aligned}$$

which is a quadratic in t. We can rewrite this in the form of $f(t) > 0$:

$$-(\Delta\mathbf{x}_2 - \Delta\mathbf{x}_1) \cdot (\Delta\mathbf{x}_2 - \Delta\mathbf{x}_1)t^2 - 2(\mathbf{x}_2^s - \mathbf{x}_1^s) \cdot (\Delta\mathbf{x}_2 - \Delta\mathbf{x}_1)t + (r_1 + r_2)^2 - (\mathbf{x}_2^s - \mathbf{x}_1^s) \cdot (\mathbf{x}_2^s - \mathbf{x}_1^s) > 0. \quad (1.7)$$

We also know that the relative velocity $\mathbf{r}(\Delta\mathbf{x})$ is $\Delta\mathbf{x}_1 - \Delta\mathbf{x}_2$, so the second inequality is just

$$\begin{aligned}
(\Delta\mathbf{x}_1 - \Delta\mathbf{x}_2) \cdot \mathbf{n}(t) &> 0 && (1.8) \\
(\Delta\mathbf{x}_1 - \Delta\mathbf{x}_2) \cdot (\mathbf{x}_2^s + t\Delta\mathbf{x}_2 - \mathbf{x}_1^s - t\Delta\mathbf{x}_1) &> 0 && (1.9) \\
(\Delta\mathbf{x}_1 - \Delta\mathbf{x}_2) \cdot [(\mathbf{x}_2^s - \mathbf{x}_1^s) + t(\Delta\mathbf{x}_2 - \Delta\mathbf{x}_1)] &> 0 && (1.10) \\
(\Delta\mathbf{x}_1 - \Delta\mathbf{x}_2) \cdot (\Delta\mathbf{x}_2 - \Delta\mathbf{x}_1)t + (\Delta\mathbf{x}_1 - \Delta\mathbf{x}_2) \cdot (\mathbf{x}_2^s - \mathbf{x}_1^s) &> 0. && (1.11)
\end{aligned}$$

If both polynomial inequalities are satisfied for some t with $0 \leq t \leq 1$, then a collision has occurred at this time step.

Implementation. There are multiple ways of implmenting this particle-particle continuous collision detection algorithm. You can follow the formula and algorithm described above, and return the earlist t_c (see geometry.collision.continuous_collide_particle_particle in the project code). The algorithm can also be implemented in an ad-hoc (but simpler) way. Here is a hint: Notice that (**??**) is a quadratic equation which has at most two roots. If the equation has less than two roots, will there be a continuous collision? If the equation does have two roots, let t_0 denote the smaller root, and t_1 denote the larger root. At $t < t_0$, do the two particles overlap or not? In what time interval, do the two particles overlap? Does the relative normal velocity at t_0 gauranteed to be positive or negative (so you don't have to check (**??**))? Think about these questions may lead you a simpler implementation. To make sure your implementation is robutst, you should also consider corner cases such as one shown in Fig. **??**.

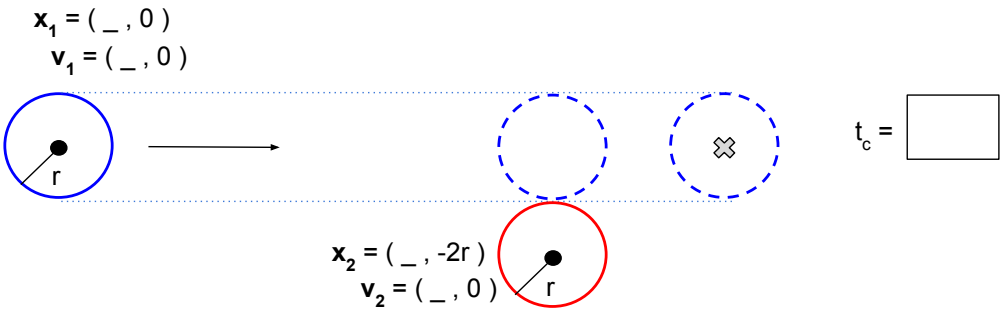

Figure 1.7: Is it important to also consider tricky scenarios, such as a case shown here. In this case, is there a valid continuous collision?

Particle–Half-plane

Since particle–half-plane collisions involve only three degrees of freedom (the position of the particle), its formulas are the simplest to derive. I implemented the detection algorithm in continuous_collide_particle_plane method in nemo/geometry/collision.py. Read and understand that method, which will help you implement your continuous_collide_particle_particle method.

1.4 Handling Multiple Simultaneous Collisions

So far we have been assuming that doing one pass of pairwise collision detection, followed by collision response in the form of applying impulses to each pair of colliding objects, is enough to produce a collision-free simulation. This assumption neglects to take into account the possibility of three of more objects colliding *simultaneously* during a single time step. Figure **??** shows one example where this assumption breaks down: only one collision is detected at first, between particles 1 and 2, but resolving this collision introduces a new one, between particles 2 and 3. Unless we do a second round of collision detection and response, we will have failed to stop this second collision, and the time step will end with the simulation in an interpenetrated state.

One way to handle the problem of simultaneous collisions is to wrap a loop around detection and response, and keep modifying positions and velocities until there are no collisions detected at the end of the time step. Here is the algorithm, a modification of that given in the previous section:

1. Detect *instantaneous* collisions at the beginning of the timestep.
2. Step the positions $\mathbf{q}^s$ and velocities $\dot{\mathbf{q}}^s$ forward using the numerical integrator to get initial *predicted* end-of-time-step positions $\mathbf{q}_0^e$ and velocities $\dot{\mathbf{q}}_0^e$. In this time integration, take into account penalty-based collision forces to try to resolve collisions.
3. For $i = 0, 1, \ldots$ until no collisions are detected or a maxium number of iterations is reached:
4. (a) Assume the particles move with constant velocity from $\mathbf{q}^s$ to $\mathbf{q}_i^e$, and perform continuous-time collision detection with $\mathbf{q}(t) = \mathbf{q}^s + t\Delta\mathbf{q}_i$, where $\Delta\mathbf{q}_i = \mathbf{q}_i^e - \mathbf{q}^s$.
   (b) Apply impulses to the predicted velocities $\dot{\mathbf{q}}_i^e$ to get new predicted velocities $\dot{\mathbf{q}}_{i+1}^e = \dot{\mathbf{q}}_i^e + \mathbf{M}^{-1}\mathbf{I}$. The vector $\mathbf{I}$ is the impulses derived in Sec. **??** to resolve the detected collisions.
   (c) Also modify positions to get new predicted positions: $\mathbf{q}_{i+1}^e = \mathbf{q}_i^e + h\mathbf{M}^{-1}\mathbf{I}$.

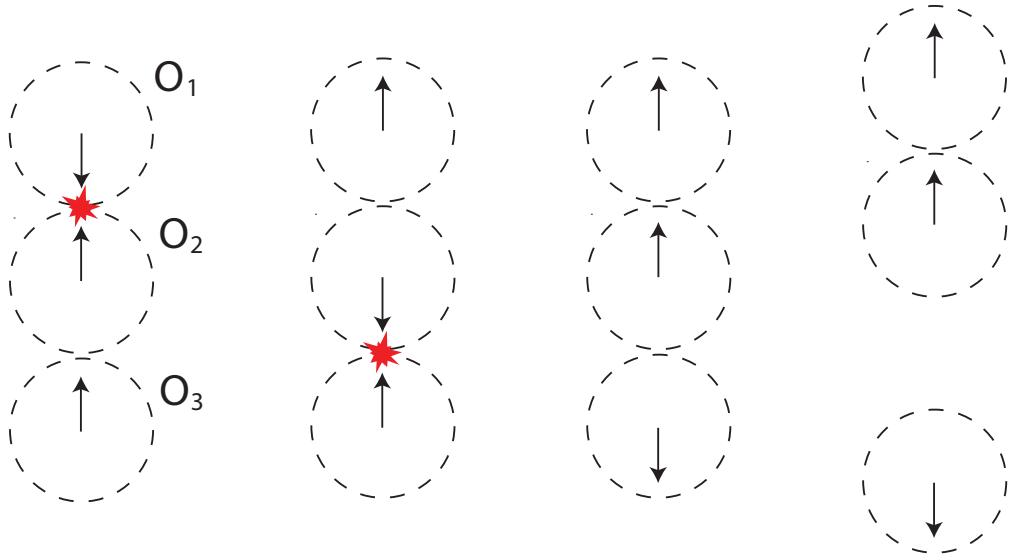

Figure 1.8: An example showing that a single round of collision detection and response can be insufficient. Initially, particles 1 and 2 are detected to be colliding (left); applying impulses resolves this collision, but introduces a new collision between particles 2 and 3 (middle left). A second iteration of collision detection and response (middle right) is necessary to produce a collision-free simulation at the end of the time step (right).

If the above loop terminates before running out of iterations, then it is guaranteed by construction to result in end-of-time-step position that are collision-free. Unfortunately, it can be shown that for some values of COR, there exist simulations for which infinitely many iterations are necessary to resolve all collisions: every impulse that stops one pair of objects from colliding causes a different pair of objects to collide. Moreover, you might need many, many iterations to fully resolve all collisions in a simulation with a large clump of objects. Even if the loop terminates, it might do so after running for a prohibitively long time. These potential pitfalls motivate us to look for an alternative approach that is guaranteed to handle multiple simultaneous collisions quickly.

Implementation details. There is a subtle detail when implmenting this iterative collison response algorithm. Step 4-(a) may return multiple continuous collisions. Then, in 4-(b), you can choose to apply impulses at those contacts *simultaneously* or *sequentially*. If you apply them *simultaneously*—meaning you compute impulses of those collisions independently and apply them to the objects—you may found that the objects sometimes become too energetic. Therefore, we suggest that you apply the impulses sequencially—applying the impulses in a for loop. In this way, the impulses applied by earlier contacts (in the for loop) may affect the object velocity such that a later contact become receding (i.e., relative normal velocity becomes positive). You need to check if a contact has became receding and ignore the contact in that case. In this approach, you avoid applying extral impulses (and thus additional energy) to the system.

1.4.1 Geometric Collision Response (Bonus)

Suppose we have some number of particles and edges that we know are about to collide with each other. There's a very simple thing we can do that is guaranteed to stop all collisions: we can simply freeze all the particles and edges in place. This solution is obviously extremely simplistic; it doesn't conserve momentum, for instance.

A more sophisticated approach is to treat the set of particles and edges as a *rigid body*: suppose every particle and edge endpoint is connected to every other one by a rigid metal beam. Since distances between

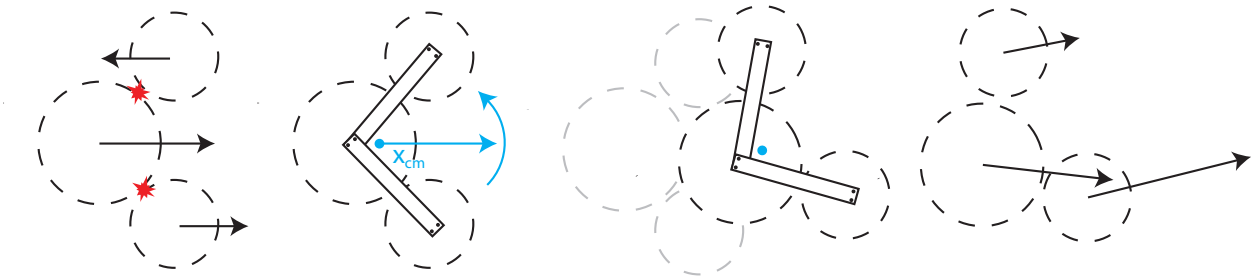

Figure 1.9: An illustration of the geometric collision response algorithm. Given a set of colliding vertices (left), we convert them into a rigid body (middle left), step the rigid body forward to the end of the time step (middle right), then convert the rigid body back into individual particles (right). The resulting position are guaranteed to be collision-free.

particles and edges cannot change, the objects cannot collide. What we need to do, then, is calculate how the rigid body would move after the beams are attached, as a function of how the individual objects moved before attachment. For the purposes of this method, we will assume that the mass of an edge is lumped at its endpoints (so that the span between the endpoints is massless.) This assumption allows us to ignore the edge itself in what follows, and work only with particles and edge endpoints (which are also particles.)

Suppose we have a collection of particles with start-of-time-step positions $\mathbf{x}_i^s$ and (colliding) end-of-time-step positions $\mathbf{x}_i^e$. We also suppose that the particles move from their old to new positions with constant velocity $\Delta\mathbf{x}_i = \mathbf{x}_i^e - \mathbf{x}_i^s$. We want to do the following:

1. Treat the particles as if they were part of a rigid body; that is, treat them as if we connected them with rigid beams at the start of the time step.

2. Step the rigid body forward in time to the end of the time step.

3. Set each particle's modified end-of-time-step position $\mathbf{x}_i^m$ to the position dictated by the motion of the rigid body.

4. Also set the particle's modified end-of-time-step velocity to $(\mathbf{x}_i^m - \mathbf{x}_i^s)/h$, where h is the length of the time step.

Figure **??** shows this algorithm step by step.

If the masses of the particles are m_i, their *center of mass* is given by

$$\mathbf{x}_{cm}^s = \frac{\sum_i m_i \mathbf{x}_i^s}{\sum_i m_i}.$$

When all masses are equal, the center of mass is just the average of the particle positions. If masses are unequal, the center of mass is a weighted average, with the more massive particles having greater weight. A rigid body's configuration is completely determined by only three degrees of freedom: the position of the center of mass of the body, and the orientation (rotation) of the body about the center of mass. Similarly, the motion of the rigid body is determined by the velocity $\Delta\mathbf{x}_{cm}$ and angular velocity (speed of rotation) ω_{cm} of the center of mass. At the start of the time step, we want to convert the individual particles into a single rigid body; we do so by calculating $\Delta\mathbf{x}_{cm}$ and ω_{cm}.

To find $\Delta\mathbf{x}_{cm}$, we invoke conservation of momentum. The momentum of the rigid body has to be equal

to the momentum of the individual particles:

$$\left(\sum_i m_i\right)\Delta\mathbf{x}_{cm} = \sum_i m_i \Delta\mathbf{x}_i$$
$$\Delta\mathbf{x}_{cm} = \frac{\sum_i m_i \Delta\mathbf{x}_i}{\sum_i m_i}.$$

Similarly, ω_{cm} is derived by looking at conservation of angular momentum. The total angular momentum (about the center of mass) of the individual particles is:

$$L = \sum_i m_i(\mathbf{x}_i^s - \mathbf{x}_{cm}) \times (\Delta\mathbf{x}_i - \Delta\mathbf{x}_{cm}),$$

where $\times$ is the cross-product. We want the angular momentum of the rigid body to have the same value. Angular momentum is related to angular velocity by the formula

$$L = I\omega_{cm},$$

where I is the moment of inertia of the rigid body:

$$I = \sum_i m_i \left[\frac{2}{5}R_i^2\mathbf{Id} + A^T(\mathbf{x}_i^s - \mathbf{x}_{cm})A(\mathbf{x}_i^s - \mathbf{x}_{cm})\right], \tag{1.12}$$

where $A(\mathbf{v})$ is the cross-product matrix:

$$A(\mathbf{v}) = \begin{bmatrix} 0 & -\mathbf{v}_z & \mathbf{v}_y \\ \mathbf{v}_z & 0 & -\mathbf{v}_x \\ -\mathbf{v}_y & \mathbf{v}_x & 0 \end{bmatrix}$$

(see derivation here). Therefore, we can compute ω_{cm} using $\omega_{cm} = L/I$.

Now we know how to step the rigid body forward in time: the center of mass translates by $\Delta\mathbf{x}_{cm}$, and the body rotates about the center of mass by ω_{cm}. To convert the rigid body's new position into new positions for the individual particles, we use the formula

$$\mathbf{x}_i^e = \mathbf{x}_{cm} + \Delta\mathbf{x}_{cm} + R_{\omega_{cm}}(\mathbf{x}_i^s - \mathbf{x}_{cm}) \tag{1.13}$$

where $R_{\omega_{cm}}$ is rotation about the origin by ω_{cm}.

Handling Fixed Objects

There is another wrinkle that must be addressed: if the set of colliding objects includes a fixed object (such as a fixed particle, an edge containing an endpoint that's fixed, or any half-plane), we cannot move the set of objects as a rigid body since the fixed object's position is not allowed to change. Thus, when applying geometric collision response to a set of objects that includes a fixed object, instead of the above set $\mathbf{x}_i^e = \mathbf{x}_i^s$ and $\mathbf{v}_i^e = 0$. There are more clever things you could do (for instance, if the only fixed object is a particle, you might set that particle's position to be the center of mass, and allow the other particles to rotate about that position) but they are beyond the scope of this course note.

Impact Zones

In the previous section we saw how to take a set of colliding objects and compute end-of-time-step positions for them that are guaranteed to be collision-free, by treating them all as part of one rigid body. What we still need is the big picture: given some start-of-time-step positions $\mathbf{q}^s$ and end-of-time-step positions $\mathbf{q}^e$, with some of the objects colliding as they move between these two positions, to which subset of the objects

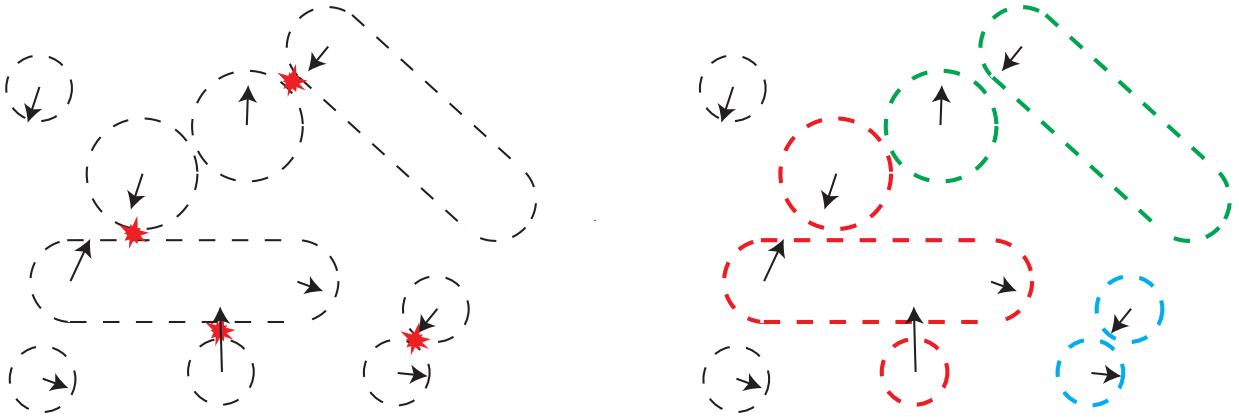

Figure 1.10: A number of collisions are detected during a time step (left). Based on these detected collisions, the particles in the simulation involved in collisions are separated into impact zones (right). Each impact zone is a different color.

do we apply the geometric response discussed in section **??**? Moving *all* particles and edges in the simulation as a rigid body, even those not at all involved in a collision, is clearly a poor solution. Treating all objects involved in a collision as part of the same rigid body is also suboptimal; if there are two clumps of collisions in two separate areas of the simulation, we don't want to couple them unnecessarily by gluing the two clumps together into one rigid body.

Instead, we will split up all particles involved in a collision into one or more *impact zones*. An impact zone is a set of particles, as well as a boolean flag indicating whether or not a half-plane is colliding with one of the particles in the impact zone.[1] Conceptually, each impact zone is one clump of interconnected collisions, with no particle shared by more than one impact zone. Each impact zone will be treated as a separate rigid body during geometric collision response; figure **??** illustrates the impact zones for an example set of detected collisions.

Here is the concrete algorithm for how to turn a set of detected collisions into disjoint impact zones:

1. First, create an impact zone for each detected collisions.
   (a) For each collision detected between particles i and j, create an impact zone containing the set of particles $\{i, j\}$. This impact zone does not involve a half-plane.
   (b) For each collision detected between particle i and edge whose endpoints are particles j and k, create an impact zone containing the set of particles $\{i, j, k\}$. This impact zone does not involve a half-plane.
   (c) For each collision detected between a particle i and a half-plane, create an impact zone containing only the lone vertex $\{i\}$. This impact zone does involve a half-plane.
2. Two impact zones are not allowed to share the same particle. If any two impact zones Z_1 and Z_2 share a particle, they must be *merged*: replace the two zones with a single new zone containing the union of Z_1 and Z_2's particles. The new zone involves a half-plane if either of the old zones did.
3. Repeat the previous step until all zones are disjoint.

Once we have a set of disjoint impact zones, we can adjust the end-of-time-step positions of the vertices in each zone using geometric collision response. It is important to stress that this response is applied to each zone separately: each zone acts as a different rigid body.

[1] We need to keep track of which impact zones involve a half-plane since we must treat such zones specially. See section **??**.

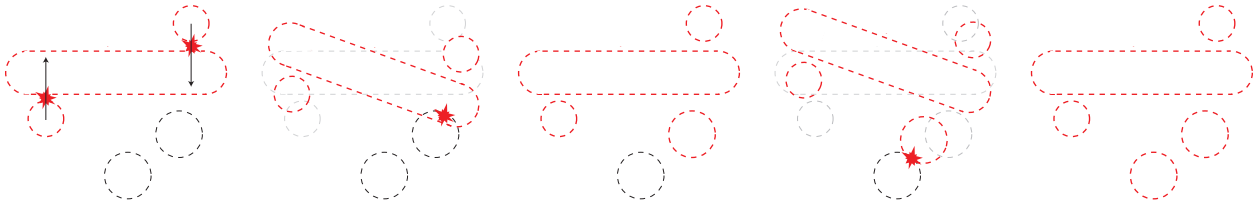

Figure 1.11: Iterative geometric collision handling. Some collisions are detected, and impact zones constructed (left). Stepping the simulation forward causes new collisions (middle left), so the impact zones grow (middle). This process repeats a second time (middle right and right).

Just as when resolving collisions using impulses, there is a possibility that one iteration of response could cause new collisions: a zone Z_1 might, while moving as a rigid body, collide into an object not in Z_1. One possibility is to go back to the approach in Sec. **??** and apply impulses between the rigid body and the new object. For now, if such a collision is detected, we will *grow* Z_1 by adding the new object to it, and recomputing the end-of-time-step positions for the particles in the new Z_1. As impact zones grow in this way, it is also possible for them to merge. Here is the algorithm in more detail; it assumes we have start-of-time-step positions and velocities $\mathbf{q}^s, \dot{\mathbf{q}}^s$ and predicted end-of-time-step positions and velocities $\mathbf{q}^e, \dot{\mathbf{q}}^e$, and are trying to find collision-free, modified end-of-time-step positions and velocities $\mathbf{q}^m, \dot{\mathbf{q}}^m$.

1. Perform continuous-time collision detection using positions $\mathbf{q}^s$ and $\mathbf{q}^e$.
2. Initialize $\mathbf{q}^m = \mathbf{q}^e$ and $\dot{\mathbf{q}}^m = \dot{\mathbf{q}}^e$.
3. Construct a list of disjoint impact zones $\mathbf{Z}$ from the detected collisions.
4. For each impact zone in $\mathbf{Z}$, apply geometric collision response, using positions $\mathbf{q}^s$ and $\mathbf{q}^m$, and modifying $\mathbf{q}^m$ and $\dot{\mathbf{q}}^m$ for the vertices in those zones.
5. Perform continuous-time collision detection using positions $\mathbf{q}^s$ and $\mathbf{q}^m$.
6. Construct a new list of impact zones $\mathbf{Z}'$ consisting of all impact zones in $\mathbf{Z}$, plus one zone for each detected collision.
7. Merge the zones in $\mathbf{Z}'$ to get disjoint impact zones.
8. If $\mathbf{Z}$ and $\mathbf{Z}'$ are equal, the algorithm is done: $\mathbf{q}^m$ and $\dot{\mathbf{q}}^m$ are the new, collision-free end-of-time-step positions. $\mathbf{Z}$ and $\mathbf{Z}'$ are equal if they contain exactly the same impact zones; impact zones are the same if they contain the same particles and they both involve, or both don't involve, a half-plane. If $\mathbf{Z} \neq \mathbf{Z}'$, go to step 9.
9. Set $\mathbf{Z} = \mathbf{Z}'$ and go to step 4.

Figure **??** illustrates the algorithm. Unlike the interactive impulse algorithm described in section **??**, this algorithm is guaranteed to terminate after finitely many iterations: during each iteration, either an impact zone grows, or an impact zone that didn't involve a half-plane now involves a half-plane. This process must eventually stop (in the worst case, with every single particle contained in one impact zone that involves a half-plane).

Putting It All Together: Hybrid Collision Handling

Although geometric collision response is guaranteed to produce collision-free end-of-time-step positions, and conserves momentum and angular momentum, it is important to realize that it is not physically correct: for instance, a particle that hits a fixed edge at an angle should either reflect off of the edge (for COR = 1.0),

slide along the edge (for COR = 0.0), or reflect at some angle between these two extremes. In every case, the particle's motion in the direction *tangential* to the edge is unrestricted. When using geometric response, on the other hand, the particle will come to a dead stop. In general, handling collisions using geometric response alone results in simulations that look "chunky" and unnatural.

However, geometric response can be a potent tool when combined with iterated impulses. Instead of iterating applying impulses until no collisions are detected – which could take a very long time – we cap the loop at a certain fixed *maximum number of iterations* n. After n iterations, if there are still unresolved collisions, we switch to geometric impulses as a failsafe. Here is the algorithm:

1. Perform collision handling using iterative impulses, as described in **??**. Stop after n iterations, or if there are no new detected collisions. The outputs from this process are new predicted end-of-time-step positions and velocities $\mathbf{q}_n^e$ and $\dot{\mathbf{q}}_n^e$.

2. If there were still unresolved collisions, perform geometric collision handling, using these new predicted quantities. The outputs from this step are guaranteed collision-free positions and velocities $\mathbf{q}^m$ and $\dot{\mathbf{q}}^m$.

If objects in a simulation become clumped in very close proximity, many simultaneous collisions occur and it becomes likely that the failsafe (step 2) is necessary to resolve these collisions. To try to create some breathing room between nearby objects and prevent this situation from occurring too often, we will add a final ingredient to our hybrid method: a weak penalty force that repels objects that are near-touching.

Further Reading

The hybrid method presented above is adapted from algorithms described in the following papers for handling collisions between cloth and objects in 3D simulations. Over the course of this theme, you have learned many of the main ideas that make up these algorithms, whose variants are still used by major movie studios today!

- Robust Treatment of Collisions, Contact and Friction for Cloth Animation. R. Bridson, R. Fedkiw and J. Anderson, ACM Transactions on Graphics, vol 21, no 3, Proc. ACM SIGGRAPH 2002, pp. 594–603.

- Robust Treatment of Simultaneous Collisions. D. Harmon, E. Vouga, R. Tamstorf and E. Grinspun, ACM Transactions on Graphics, vol 27, no 3, Proc. ACM SIGGRAPH 2008, pp. 1–4.

Chapter 2

Assignment Requirements

The core mission in this assignmenet is to add two new solvers, ContactPenaltySolver and MultiContactSolver. We will start with some collision detection algorithms.

2.1 Primitive Collision Detection

1. Implement instantaneous collision detection between particles and between particle and half-space plane. In particular, collide_plane_sphere(...) and collide_particle_particle(...) in nemo/geometry/collision.py. Please read the comments of those methods carefully.
2. Implement continuous collision detection between two particles, namely the continuous_collide_particle_plane(...) in nemo/geometry/collision.py.

At this point, you may want to sanity check your implmentation by running unit tests:

```
pytest src/tests/test_collision.py
```

and pass the tests.

2.2 Instantaneous Collision Detection

Next, implment CollisionDetector.instantaneous_contacts(...) method. This method detect collisions between particles and between a particle and a half-space plane. This first part of the method is already given, as an example to show how to detect collisions between a particle and a half-space plane and how to organize the data structure. You need to implment the part that detects particle-particle collisions. To get full credits of this part, your algorithm complexity should be at least $O(N \log N)$ where N is the number of particles.

2.3 Penalty-based Collision Response

Then, implement a penalty-based collision solver in nemo/solvers/contact_penalty_solver.py. Instead of leaving you a fully empty step function there, I left the structure of code that I implemented to give you a sense of the high-level algorithm. You could start the chunk about the response of fixed-shape–particle collisions:

```
...
elif contacts.contact_type[i] == ContactType.FIXED_SHAPE_PARTICLE:
    shape_id = contacts.contact_instance0[i]
    par_id = contacts.contact_instance1[i]
    depth = np.dot(contacts.contact_point1[i] - contacts.contact_point0[i], contacts.contact_normal[i]
```

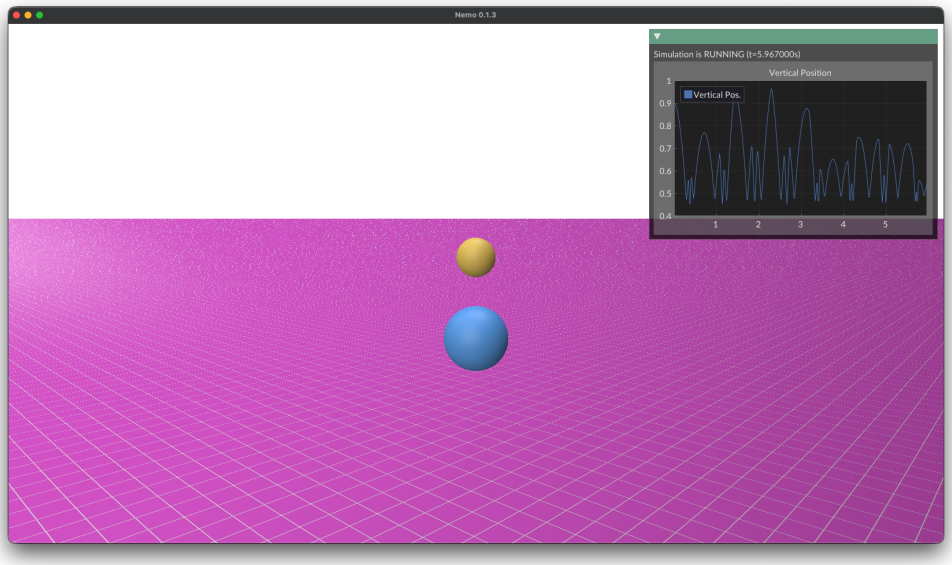

Figure 2.1: Example screenshot.

```
        if depth < 0.0:
            # TODO: implement the impulse-based collision response for particle-fixed shape contact.
            # You need to use the ‘penalty_force‘ function above to compute the penalty force.
            # and update f_penalty accordingly.

            # Replace the following line with your implementation.
            pass
    ...
```

and then work on particle–particle response under the line

```
    if contacts.contact_type[i] == ContactType.PARTICLE_PARTICLE:
        ...
```

At this point, if your implmentation is correct, you should be able to test it by specifying the solver type using

```
solver:
  type: contact_penalty
...
```

in .YAML scene files. For example, by running

```
python -m assignments.run pa3 scenes/pa3/scene01.yml
```

you should see spheres bouncing around (see Fig. **??**).

2.4 Continous Collision Detection

Implement CollisionDetector.continuous_contacts(). This should be a relatively straight forward implementation. All you need to do is to use continuous_collide_particle_plane and continuous_collide_particle_particle and fill the data structure in sim.contacts.Contacts.

2.5 Multi-Contact Solver

The final punch is to implement MultiContactSolver as described in Sec. **??**. Note that you are not required to implment geometric collision response algorithm. In most cases, running the continuous collision response for multiple interations can resolve most of the contacts. This is especially fine when the timestep size is relatively small. In other words, in Step 3 of the algorithm in Sec. **??**, you run the iteration until either no collisions are detected or a maximum number of iteration (specified by maxits in the code) is reached.

BONUS CREDITS.

Lastly, **you can earn 10 bonus points** if you implment the geometric collision response and impact zones described above. Since the implmentation requires additional data structure, feel free to modify the starter code to organize your data structure. If you choose to take this challenge, please include a README file in your submission to tell us how/where you implmented the algorithm.